

Low-Rank Adaptation of MindEye2 for Data-Efficient fMRI-to-Image Decoding

Mohammad Amin Mohammadi Nezhad

Guilan University

Project Supervisor

Dr. Seyed Abolghasem Mirroshandel

September 2026

Abstract

Reconstructing seen images from functional MRI (fMRI) has advanced rapidly by combining pretrained vision-language embeddings with diffusion models. MindEye2 (Scotti et al., 2024) showed that a model pretrained across seven participants of the Natural Scenes Dataset (NSD) can be adapted to a new participant with as little as one hour of scanning. That adaptation step, however, updates every shared parameter of a very large network, so each new subject is expensive to train and to store. This project asks whether Low-Rank Adaptation (LoRA) can replace full fine-tuning in that step. I implemented LoRA for the shared MindEye2 backbone and output heads inside a reproducible, configuration-driven research repository (`mindeye2-lora`) that runs end-to-end on a single NVIDIA T4 GPU in Google Colab, and compared six experimental arms with three random seeds each (18 training runs of 150 epochs). The primary outcome is two-way CLIP identification accuracy on held-out NSD test images; secondary outcomes are trainable-parameter count, GPU memory, training time, and reconstructions produced by the SDXL unCLIP decoder. Across three seeds, every parameter-efficient arm outperformed full fine-tuning: LoRA at rank 4 reached 97.48 % two-way CLIP identification versus 96.21 % for full fine-tuning ($\Delta = +1.27$ points, 95 % CI +0.99 to +1.59) while training 2.56 % of the parameters and reducing per-subject storage from 2,700 MB to 71.6 MB. Accuracy was flat across ranks 4 to 64, and a frozen shared model with only the subject-specific ridge layer trained already reached 95.47 %. The report closes with a discussion of when low-rank adaptation is sufficient for new-subject decoding and what this implies for low-cost brain-decoding research.

Keywords: fMRI decoding · image reconstruction · MindEye2 · Natural Scenes Dataset · LoRA · parameter-efficient fine-tuning · diffusion models · CLIP

Contents

- 1 Introduction
 - 1.1 Motivation
 - 1.2 Research questions and hypotheses
 - 1.3 Contributions
 - 1.4 Organization of this report
- 2 Background and Related Work
 - 2.1 fMRI and visual decoding
 - 2.2 CLIP embeddings, diffusion priors and unCLIP
 - 2.3 Cross-subject brain decoding
 - 2.4 Parameter-efficient fine-tuning
- 3 The MindEye2 Architecture
 - 3.1 Overview
 - 3.2 Stage 1 — Subject-specific functional alignment
 - 3.3 Stage 2 — Shared residual MLP backbone
 - 3.4 Stage 3 — Heads
 - 3.5 From embeddings to pixels: the SDXL unCLIP decoder
 - 3.6 Training objective and protocol
- 4 Data
 - 4.1 The Natural Scenes Dataset
 - 4.2 Preprocessing
 - 4.3 Training and test splits
 - 4.4 Distribution through Hugging Face
 - 4.5 Data use and ethics
- 5 Methods: LoRA for MindEye2
 - 5.1 Low-rank adaptation
 - 5.2 Where LoRA is applied in MindEye2
 - 5.3 Experimental design
 - 5.4 Evaluation metrics
 - 5.5 Statistical analysis plan
- 6 Implementation: The mindeye2-lora Repository
 - 6.1 Design goals
 - 6.2 Architecture of the software
 - 6.3 Workflow
 - 6.4 Validation and reproducibility
- 7 Results
 - 7.1 Training dynamics
 - 7.2 Primary outcome: two-way CLIP identification
 - 7.3 Does LoRA match full fine-tuning?

7.4 Efficiency

7.5 Effect of rank

7.6 Brain-decoded images

8 Discussion

8.1 Does LoRA work well for new-subject decoding?

8.2 Practical implications

8.3 Limitations

8.4 Future work

8.5 Ethical considerations

9 Conclusion

Acknowledgements

References

1 Introduction

1.1 Motivation

Decoding what a person is seeing from measurements of their brain activity is one of the most direct tests of how well we understand visual representation in the human cortex. Early work could only *identify* which of a known set of images had been viewed (Kay et al., 2008). Over the past three years, however, the combination of large functional MRI datasets with pretrained vision–language models and generative diffusion models has made it possible to *reconstruct* natural images from fMRI with striking semantic fidelity (Takagi & Nishimoto, 2023; Ozcelik & VanRullen, 2023; Scotti et al., 2023).

A persistent limitation of these systems is their appetite for data. Most were trained separately for each participant on the Natural Scenes Dataset (NSD; Allen et al., 2022), in which each person spent up to 40 hours in a 7-Tesla scanner. Collecting that much data for every new individual is impractical outside a handful of well-funded labs. MindEye2 (Scotti et al., 2024) addressed this by pretraining a single *shared-subject* model on seven NSD participants and then fine-tuning it on a new participant, obtaining high-quality reconstructions from only one hour (one scanning session) of that person's data.

The fine-tuning step in MindEye2 is *full* fine-tuning: every parameter of the shared network is updated for the new subject. At the scale used in the original paper this is a very large network — the output projection of the backbone alone maps a 4,096-dimensional latent vector to $256 \times 1,664$ CLIP token values, which is roughly 1.7 billion weights (Section 5.2). Full fine-tuning therefore requires high-memory GPUs, produces a complete multi-gigabyte model copy for every subject, and — with only 688 training trials available in the one-hour regime — risks overfitting a very large number of parameters to very little data.

Parameter-efficient fine-tuning (PEFT) methods offer a principled alternative. Low-Rank Adaptation (LoRA; Hu et al., 2022) freezes the pretrained weights and learns a small low-rank update for selected layers. In language and vision models LoRA routinely matches full fine-tuning while training well under one percent of the parameters. Whether that holds for brain decoding — where the new subject's data differ from pretraining data not only in content but in the very anatomy that generates the signal — is an empirical question that this project sets out to answer under controlled conditions.

1.2 Research questions and hypotheses

The project is organized around three questions, each paired with a pre-stated hypothesis:

- **RQ1 — Accuracy.** Can LoRA adaptation of the shared MindEye2 network decode a new subject as accurately as full fine-tuning? *H1 (non-inferiority):* for at least one tested rank, the two-way CLIP identification accuracy of LoRA is not lower than that of full fine-tuning by more than a pre-specified margin δ .
- **RQ2 — Rank.** How does decoding accuracy depend on the LoRA rank r ? *H2:* accuracy increases with r and saturates, so that a modest rank captures most of the achievable performance.
- **RQ3 — Efficiency.** What does LoRA save in practice? *H3:* LoRA reduces trainable parameters, optimizer state and per-subject storage by one or more orders of magnitude, with smaller savings in wall-clock time.

1.3 Contributions

This work makes four contributions. First, it provides an implementation of LoRA for the MindEye2 architecture that can be switched on per component and per rank from a configuration file. Second, it delivers a complete, reproducible pipeline — data retrieval from Hugging Face, training, evaluation, statistics, image generation and reporting — that runs on a single 15 GB T4 GPU, together with a documented set of engineering solutions that make this possible. Third, it reports a controlled comparison of six arms with three seeds each and a pre-specified statistical analysis, rather than a single-run comparison. Fourth, it presents brain-decoded images for every arm, generated with the MindEye2 SDXL unCLIP decoder, so that quantitative differences can be inspected qualitatively.

1.4 Organization of this report

Section 2 reviews fMRI decoding, the vision-language and diffusion models that MindEye2 builds on, and parameter-efficient fine-tuning. Section 3 describes the MindEye2 architecture. Section 4 describes the data: NSD, its preprocessing, and its distribution through Hugging Face. Section 5 presents the methods — the LoRA formulation, where it is applied, the experimental design, metrics and statistical plan. Section 6 describes the software I built and the engineering problems solved along the way. Section 7 is reserved for results, Section 8 discusses them together with limitations and ethical considerations, and Section 9 concludes.

2 Background and Related Work

2.1 fMRI and visual decoding

Functional MRI measures the blood-oxygen-level-dependent (BOLD) signal, an indirect and temporally sluggish correlate of neural activity, in small cubes of tissue called *voxels* (1.8 mm on a side in NSD). A decoding model learns the mapping from a pattern of voxel responses back to a description of the stimulus, the reverse direction of an encoding model (Naselaris et al., 2011). Early decoders used linear models and hand-designed image features; Kay et al. (2008) showed that a viewed image could be identified among a large set of candidates, establishing the *identification* paradigm that survives today in the two-way identification metric used in this report.

Deep networks changed the target representation. Horikawa and Kamitani (2017) decoded hierarchical features of a convolutional network from fMRI and used them to identify object categories, including for imagined images. The arrival of large diffusion models then enabled full reconstructions: Takagi and Nishimoto (2023) mapped fMRI to the latent and conditioning spaces of Stable Diffusion, and Brain-Diffuser (Ozcelik & VanRullen, 2023) combined a VDVAE for low-level structure with Versatile Diffusion for semantics. MindEye (Scotti et al., 2023) mapped fMRI into CLIP image space with a large MLP trained contrastively, added a diffusion prior to align the predictions with the CLIP distribution, and obtained state-of-the-art retrieval and reconstruction on NSD.

2.2 CLIP embeddings, diffusion priors and unCLIP

CLIP (Radford et al., 2021) trains an image encoder and a text encoder jointly so that matching image-caption pairs have similar embeddings. OpenCLIP (Ilharco et al., 2021; Cherti et al., 2023) provides open reproductions at larger scales, including the ViT-bigG/14 model used by MindEye2, whose final layer produces 256 spatial tokens of 1,664 dimensions for each image. Latent diffusion models (Rombach et al., 2022) generate images by iteratively denoising in the latent space of an autoencoder; SDXL (Podell et al., 2024) is a larger, higher-resolution member of this family. DALL·E 2 (Ramesh et al.,

2022) introduced two ideas reused by MindEye: a *diffusion prior* that maps one embedding (a caption embedding in DALL-E 2, a brain embedding in MindEye) into the distribution of CLIP image embeddings, and an *unCLIP* decoder that generates an image conditioned on a CLIP image embedding rather than on text.

2.3 Cross-subject brain decoding

Because every brain differs in size, folding and functional layout, voxels do not correspond across individuals, and NSD participants mostly viewed *different* images, which rules out alignment methods that require shared stimuli. MindEye2 solves this with a subject-specific linear map into a shared latent space followed by shared non-linear layers (Section 3). Related approaches include MindBridge (Wang et al., 2024), which uses subject-specific aggregation layers and a cyclic reconstruction objective, and MindTuner (Gong et al., 2025), which fine-tunes a multi-subject model on a new subject using LoRA together with a “Skip-LoRA” variant to capture subject-specific *visual fingerprints*, and adds an fMRI-to-text alignment step. MindTuner shows that low-rank adapters are a plausible tool for new-subject decoding; the present project isolates that question by keeping the rest of the MindEye2 pipeline unchanged and comparing LoRA directly against full fine-tuning across multiple seeds.

2.4 Parameter-efficient fine-tuning

Parameter-efficient fine-tuning adapts a large pretrained model by training a small number of new or selected parameters. Adapters (Houlsby et al., 2019) insert small bottleneck layers between frozen layers; prefix and prompt tuning learn input-side vectors. LoRA (Hu et al., 2022) instead constrains the *update* to each adapted weight matrix to be low-rank. It is motivated by the empirical observation that fine-tuning updates have low intrinsic dimensionality (Aghajanyan et al., 2021). LoRA has two practical advantages over adapters: after training, the low-rank update can be merged into the frozen weight so inference costs nothing extra, and each adapted model can be stored as a compact file of low-rank factors. Extensions such as QLoRA (Dettmers et al., 2023) combine LoRA with weight quantization to fit very large models on small GPUs. For brain decoding, the storage argument is especially attractive: a single frozen base model could serve many participants, each represented by a subject-specific ridge layer and a small set of adapters.

3 The MindEye2 Architecture

3.1 Overview

MindEye2 decodes a single fMRI response into three outputs: an image reconstruction, an embedding for retrieving the viewed image from a candidate set, and a blurry low-level reconstruction that captures layout and colour. Figure 1 summarizes the pipeline. Conceptually it has three stages: a subject-specific *functional alignment* that maps each person’s voxels into a common latent space; a *shared non-linear mapping* from that space to OpenCLIP ViT-bigG/14 image tokens; and a set of *heads* that turn those tokens into images and embeddings. The key design choice is that everything after the first linear layer is shared across participants, which is what allows knowledge learned from seven people to transfer to an eighth.

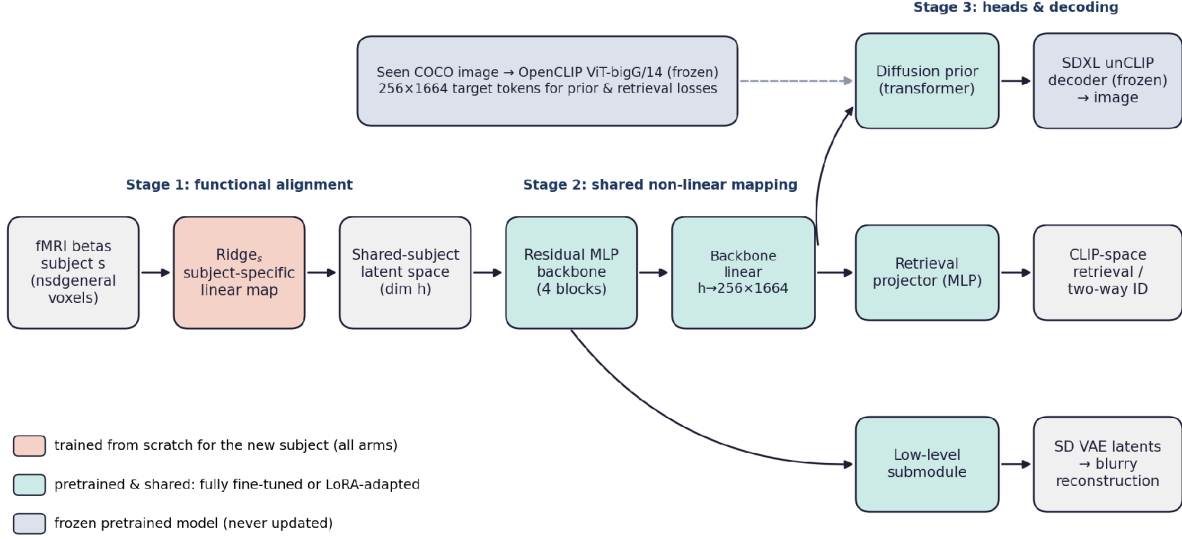


Figure 1. Overview of MindEye2 as used in this project. Colours indicate how each component is treated when adapting to a new subject: the subject-specific ridge layer (orange) is always trained from scratch; the shared, pretrained components (teal) are either fully fine-tuned or adapted with LoRA depending on the experimental arm; the CLIP image encoder and the SDXL unCLIP decoder (blue-grey) are frozen throughout. The low-level branch may be disabled in lightweight configurations.

3.2 Stage 1 — Subject-specific functional alignment

Each participant's response to an image is a vector of single-trial *beta* values, one per voxel inside the *nsdgeneral* region of interest, a mask covering the visually responsive posterior cortex. The number of voxels differs by participant (15,724 for subject 1). A subject-specific linear layer, which the authors call a ridge-regression mapping because it is trained with weight decay, projects this vector into a shared latent space of dimension h (4,096 in the paper; 1,024 in this project's T4 configuration). No shared stimuli are needed: the alignment emerges because all subjects' ridge layers feed the same downstream network and are trained jointly against the same CLIP targets. When a new subject is added, a new ridge layer must be learned from scratch, because its input dimension and voxel identities are unique to that person.

3.3 Stage 2 — Shared residual MLP backbone

The shared latent vector is processed by a residual MLP backbone of four blocks, each consisting of an $h \times h$ linear layer, layer normalization, a GELU non-linearity and dropout, with a skip connection around the block. A final linear layer, the *backbone linear* projection, maps the h -dimensional output to $256 \times 1,664 = 425,984$ values, reshaped into the same 256-token format produced by the ViT-bigG/14 image encoder. This projection contains the overwhelming majority of the model's parameters and is therefore the most consequential place to apply LoRA (Section 5.2).

3.4 Stage 3 — Heads

Diffusion prior

The backbone's output tokens are semantically informative but lie off the true distribution of CLIP image embeddings. Following DALL·E 2, MindEye2 trains a transformer-based *diffusion prior* that, conditioned

on the backbone output, iteratively denoises a random tensor into a $256 \times 1,664$ embedding that resembles a real ViT-bigG/14 image embedding. Its training loss is the mean-squared error between the denoised prediction and the true CLIP tokens of the viewed image.

Retrieval projector

A second head, a small MLP projector, maps the backbone tokens into a space optimized for contrastive retrieval. It is trained with a contrastive loss that pulls each brain embedding towards the CLIP embedding of its own image and away from the other images in the batch. As in the original MindEye, the loss begins as BiMixCo, a bidirectional contrastive loss with mixup augmentation of the fMRI inputs, for the first third of training, and then switches to SoftCLIP, which uses soft targets derived from CLIP image-image similarities.

Low-level submodule

The third head predicts the latent representation of a Stable Diffusion variational autoencoder, which is decoded into a blurry image that preserves layout, colour and coarse shape. In the original work this blurry image is used to initialize the final image-to-image refinement. The low-level branch is optional and is often disabled in reduced-memory configurations.

3.5 From embeddings to pixels: the SDXL unCLIP decoder

To turn predicted CLIP tokens into an image, the MindEye2 authors fine-tuned SDXL so that it accepts the $256 \times 1,664$ ViT-bigG/14 image tokens as its conditioning input in place of text embeddings, producing an *unCLIP* model. Given a predicted embedding from the diffusion prior, this decoder generates an image whose content matches the embedding. In the full MindEye2 pipeline an optional refinement stage follows: a caption is predicted from the brain embedding with a GIT captioning model (Wang et al., 2022), and the unCLIP output is refined with the base SDXL model through image-to-image diffusion guided by that caption. The decoder and all image models are frozen; they are never updated during subject adaptation. In this project the decoder is used to generate sixteen reconstructions per experimental arm unrefined unCLIP outputs, without the GIT-caption refinement stage.

3.6 Training objective and protocol

The total training loss is a weighted sum of the three head losses: the diffusion-prior loss scaled by `prior_scale`, the contrastive retrieval loss scaled by `clip_scale`, and, when enabled, the low-level loss scaled by `blur_scale`. Pretraining samples batches equally from seven NSD participants and uses all 40 sessions of each; the resulting checkpoint is then fine-tuned on the held-out eighth participant. Table 1 lists the paper-scale settings documented in the public MindEye2 repository.

Table 1. Paper-scale MindEye2 training settings, as documented in the public MindEye2 repository.

Setting	Value	Meaning
<code>hidden_dim</code>	4,096	dimension h of the shared latent space
<code>n_blocks</code>	4	residual MLP blocks in the backbone
<code>num_epochs</code>	150	training epochs
<code>batch_size</code>	42	samples per batch
<code>max_lr</code>	3×10^{-4}	peak learning rate
<code>mixup_pct</code>	0.33	fraction of training using BiMixCo before SoftCLIP
<code>prior_scale / clip_scale / blur_scale</code>	30 / 1 / 0.5	loss weights for prior, retrieval and low-level losses

Setting	Value	Meaning
mixed precision	fp16	training numerics

4 Data

4.1 The Natural Scenes Dataset

The Natural Scenes Dataset (Allen et al., 2022) is the largest fMRI dataset of its kind and the standard benchmark for image reconstruction from brain activity. Eight healthy adults were scanned at 7 Tesla while viewing natural photographs drawn from the Microsoft COCO dataset (Lin et al., 2014). Each participant took part in up to 40 scanning sessions of roughly one hour, each session containing about 750 image presentations. Every image was shown for three seconds and repeated three times over the course of the experiment, while participants performed a continuous recognition task (reporting whether they had seen the image before) to keep them attentive. Table 2 summarizes the properties of NSD that matter for this project.

Table 2. Key properties of the Natural Scenes Dataset relevant to this project.

Property	Value
Participants	8 (subj01–subj08)
Scanner / resolution	7 T; 1.8 mm isotropic voxels for the functional data used here
Sessions	up to 40 per participant (~1 hour each); subjects 1, 2, 5 and 7 completed all 40
Trials per session	~750 image presentations
Stimuli	~73,000 distinct COCO images; each participant was assigned ~10,000
Repetitions	each image presented 3 times, 3 s per presentation
Shared images	1,000 images (“shared1000”) seen by every participant, used as the test set
Task	continuous recognition (“have you seen this image before?”)
Response estimates	single-trial betas estimated with GLMsingle

4.2 Preprocessing

MindEye2 does not use raw fMRI time series. It uses single-trial response amplitudes (*betas*) estimated by GLMsingle (Prince et al., 2022), which fits a hemodynamic response function per voxel, removes noise with data-driven regressors and applies ridge regularization to obtain more reliable single-trial estimates. The betas are restricted to the `nsdgeneral` mask and normalized; the MindEye2 authors distribute these as one preprocessed file per subject. For training, every presentation is treated as a separate sample, so the three repetitions of an image act as natural noise augmentation. For testing, the responses to the repetitions of each shared image are averaged, which substantially raises the signal-to-noise ratio of the test inputs.

4.3 Training and test splits

The test set consists of the shared images that the target participant viewed, with repetitions averaged; for participants who completed all 40 sessions this is the full set of 1,000 shared images, and none of these images are used for training. The training set for fine-tuning is drawn from the target participant’s non-shared trials. This project follows MindEye2’s *one-hour* regime: the target participant is subject 1

and only the first 1 session(s) of that participant's data are used for adaptation, which corresponds to roughly 688 training trials. The pretrained checkpoint was trained without any data from the target participant, so the test images and the participant's brain are both new to the model.

4.4 Distribution through Hugging Face

The full NSD is hosted by its creators and requires agreeing to the dataset's terms of use. The MindEye2 authors additionally publish their preprocessed training data, pretrained models and evaluation assets as a Hugging Face dataset repository, `pscott/mindeyev2`, which is the data source for this project. The complete repository is large (on the order of 215 GB), because it contains data for all eight participants and many model checkpoints. Table 3 lists the categories of files it contains and how this project uses them.

Table 3. Contents of the `pscott/mindeyev2` Hugging Face repository used in this project (file names as in the upstream repository at the time of writing).

Asset	Content	Use in this project
Beta files (<code>betas_all_subj0X_fp32_renorm.hdf5</code>)	preprocessed nsdgeneral betas for every trial of subject X	brain inputs for the target subject
WebDataset shards (<code>wds/subj0X/train test/*.tar</code>)	per-trial metadata: image index, trial, session and repetition indices	defines the train/test split and session selection
Stimulus images (COCO images at 224 px, HDF5)	the viewed images in index order	computing CLIP targets and ground-truth panels
Pretrained checkpoints (<code>train_logs/...</code>)	multi-subject models, one per held-out subject	initialization for all arms
unCLIP checkpoint	SDXL fine-tuned to accept ViT-bigG/14 tokens	image generation (frozen)
Auxiliary models	autoencoder for the low-level branch; bigG→ViT-L converter for captioning	loaded only when the corresponding feature is enabled
External Hugging Face models	OpenCLIP ViT-bigG/14 (LAION-2B); SDXL base	CLIP targets; optional refinement

The project's `assets.py` module downloads only the subset required by a given configuration — the target subject's beta file and shards, the image file, the chosen pretrained checkpoint and the unCLIP decoder.

4.5 Data use and ethics

NSD was collected with informed consent under institutional ethics approval, and access to it is governed by the dataset's terms and conditions, which this project follows. The data are de-identified, no attempt is made to link them to individuals, and no new human data were collected for this project. Broader ethical considerations raised by brain decoding are discussed in Section 8.5.

5 Methods: LoRA for MindEye2

5.1 Low-rank adaptation

Consider a linear layer with pretrained weight matrix W_0 of shape $d_{out} \times d_{in}$. Full fine-tuning learns an arbitrary update ΔW of the same shape. LoRA instead freezes W_0 and constrains the update to be the

product of two thin matrices, B of shape $d_{out} \times r$ and A of shape $r \times d_{in}$, with rank r much smaller than either dimension (Figure 2). The adapted layer computes

$$h = W_0 x + \left(\frac{\alpha}{r}\right) \cdot B A x \quad (1)$$

where α is a fixed scaling constant that makes the effective step size roughly independent of r . A is initialized randomly and B is initialized to zero, so at the start of training the adapted model is exactly the pretrained model and adaptation departs from it smoothly. The layer's trainable parameter count falls from $d_{out} \cdot d_{in}$ to $r \cdot (d_{in} + d_{out})$. After training, the product $(\frac{\alpha}{r})BA$ can be added into W_0 , so the adapted model runs at exactly the same speed as the original; alternatively, A and B can be stored separately as a compact per-subject adapter.

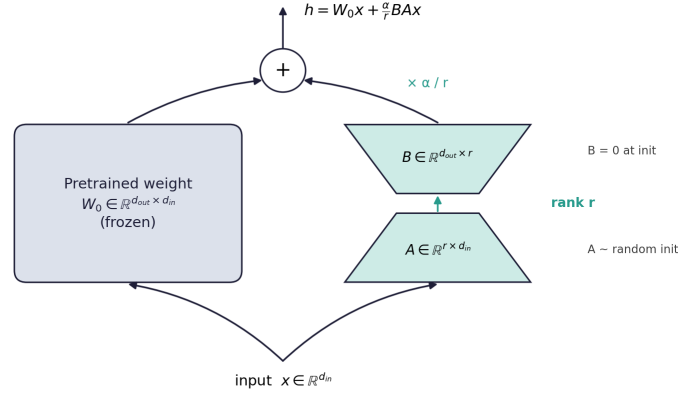


Figure 2. A LoRA-adapted linear layer. The pretrained weight W_0 is frozen; only the low-rank factors A and B (rank r) are trained, and their scaled product is added to the frozen layer's output.

5.2 Where LoRA is applied in MindEye2

Table 4 shows how each MindEye2 component is treated in the full fine-tuning arm and in the LoRA arms. Two components are handled identically in every arm. The subject-specific ridge layer is always trained from scratch, because a new participant's voxels have no pretrained counterpart. The frozen image models — the OpenCLIP encoder that provides targets and the SDXL unCLIP decoder — are never trained. The arms therefore differ only in how the *shared* pretrained components are adapted: with unconstrained updates or with low-rank updates.

Table 4. Treatment of MindEye2 components in the full fine-tuning and LoRA arms ($h = 1,024$). Parameter counts are for the components as described in Section 3.

Component	Parameters	Full fine-tuning	LoRA arms
Ridge layer, subject 1	16.1 M	trained from scratch	trained from scratch
Residual MLP blocks ($4 \times h \times h$)	4×1.05 M	all weights trained	frozen + LoRA
Backbone linear ($h \rightarrow 256 \times 1,664$)	436.6 M	all weights trained	frozen + LoRA
Diffusion prior (transformer)	252.4 M	all weights trained	frozen + LoRA attention and feed-forward projections
Retrieval projector	11.2 M	all weights trained	frozen + LoRA
Low-level submodule	disabled	disabled	disabled
LayerNorm weights and biases	small	trained	frozen

Component	Parameters	Full fine-tuning	LoRA arms
OpenCLIP ViT-bigG/14 encoder	—	frozen	frozen
SDXL unCLIP decoder	—	frozen	frozen

Yellow cells must be matched to the target-module list in `Lora.py` and the arm definitions in `main.yaml`.

Table 5 makes the parameter savings concrete for the two largest kinds of layers in the backbone. The savings are largest for the backbone linear projection, whose output dimension (425,984) is enormous compared with its input dimension: at $h = 1,024$ a rank-4 adapter replaces 436.6 million trainable weights with 1.7 million, a 256-fold reduction, and even a rank-64 adapter needs only 27.3 million.

Table 5. Trainable parameters per layer under full fine-tuning and LoRA for the backbone layers at both hidden sizes used in this project.

Layer	h	Full	r = 4	r = 8	r = 16	r = 32
Residual block linear ($h \times h$)	1,024	1,049,600	8,192	16,384	32,768	65,536
Backbone linear ($h \rightarrow 425,984$)	1,024	436,633,600	1,708,032	3,416,064	6,832,128	13,664,256
Residual block linear ($h \times h$)	4,096	16,781,312	32,768	65,536	131,072	262,144
Backbone linear ($h \rightarrow 425,984$)	4,096	1,745,256,448	1,720,320	3,440,640	6,881,280	13,762,560

For comparison, the subject-1 ridge layer ($15,724$ voxels $\rightarrow h$) has 16.1 M parameters at $h = 1,024$ and 64.4 M at $h = 4,096$. It is trained fully in every arm and is not LoRA-adapted.

The note under Table 5 reveals an important structural fact about this problem. Because the ridge layer must be learned in full, it sets a floor on the number of trainable parameters that LoRA cannot lower. Counting only the ridge layer, the four residual blocks and the backbone linear at $h = 1,024$, full fine-tuning trains about 456.9 million parameters, whereas rank-8 LoRA trains about 19.6 million (4.3 %), of which 82 % belong to the ridge layer. At paper scale ($h = 4,096$) the corresponding figures are 1.88 billion versus 68.1 million (3.6 %), with the ridge accounting for 95 %. The low-rank adapters themselves are small at both scales (3.5–3.7 million parameters); the practical cost of adding a subject to a LoRA-adapted MindEye2 is dominated by its alignment layer, which motivates the future work on low-rank alignment in Section 8.4.

5.3 Experimental design

The primary experiment (`main.yaml1`) compares six arms, each trained with three random seeds for 150 epochs, giving 18 training runs. All runs start from the same pretrained multi-subject checkpoint, use the same target participant, sessions and test set, and share the optimizer, learning-rate schedule, batch size, loss weights and number of epochs. The same three seeds are used in every arm, so that seed k determines the same data order and initialization of the new ridge layer across arms; this makes the comparison *paired* at the seed level as well as at the test-image level. The learning rate is 3×10^{-4} for full fine-tuning and 1×10^{-3} for the parameter-efficient arms, because low-rank updates have far fewer effective degrees of freedom and are undertrained at the full fine-tuning rate; this is a deliberate deviation discussed in Section 8.3. Table 6 defines the arms.

Table 6. Experimental arms of the primary experiment (6 arms $\times$ 3 seeds $\times$ 150 epochs).

Arm	Description	Shared components	Rank r	α
full	Full fine-tuning (reference)	all weights trained	—	—
lora_r4	LoRA, low rank	frozen + LoRA	4	8
lora_r16	LoRA, medium rank	frozen + LoRA	16	32

Arm	Description	Shared components	Rank r	α
lora_r64	LoRA, high rank	frozen + LoRA	64	128
bitfit	BitFit: biases and LayerNorms	frozen + LoRA	---	---
frozen	Frozen shared model, ridge layer only (lower bound)	frozen	—	—

Replace the yellow entries with the arm definitions in `main.yaml`.

Two further configurations complement the primary experiment (Table 7). `smoke.yaml` runs the entire pipeline with drastically reduced settings and exists purely to catch integration errors before expensive runs. `paper_scale.yaml` uses the paper's hidden size of 4,096, which does not fit on a T4, and is run on an A100 GPU. Its scope is deliberately limited to *efficiency* measurements — trainable parameters, peak memory and time per epoch — so that the report can state what LoRA saves at the scale of the original paper without claiming paper-scale accuracy results that were not measured.

Table 7. Configurations used in the project.

Config	Purpose	h	Runs	Epochs	GPU	Reported outputs
<code>smoke.yaml</code>	integration test	1,024	3 arms x 1 seed	10	T4	none (pipeline check)
<code>main.yaml</code>	primary experiment	1,024	6 arms x 3 seeds	150	T4 (15 GB)	accuracy, statistics, images, efficiency
<code>paper_scale.yaml</code>	paper-scale efficiency	4,096	not run	not run	A100	efficiency only

5.4 Evaluation metrics

Primary metric: two-way CLIP identification

For each of the N test images, the trained model produces a predicted CLIP embedding $\hat{y}_i$ from the averaged brain response, and the true image has a ground-truth embedding y_i computed with the frozen OpenCLIP encoder. Using a similarity function s (Pearson correlation cosine similarity), the model is said to correctly identify image i against distractor j if its prediction is more similar to the true embedding than to the distractor's. Averaging over all $N - 1$ distractors gives a per-image score, and averaging over images gives the metric:

$$TW_i = \frac{1}{(N-1)} \cdot \sum_{j \neq i} 1[s(\hat{y}_i, y_i) > s(\hat{y}_i, y_j)] \quad (2)$$

$$two_way_clip = 100 \% \cdot \frac{1}{N} \cdot \sum_i TW_i \quad (3)$$

Chance performance is 50 % and perfect performance is 100 %. The metric is computed in embedding space rather than on decoded images, because decoding every test image for all 18 runs with SDXL is not feasible on a T4; this is the same *identification* logic used by Kay et al. (2008) and by the CLIP two-way metric of the MindEye papers the ViT-bigG/14 token embedding space, after the diffusion prior.

Why retrieval percentile is not reported separately

If $rank_i$ denotes the position of the true image among all N candidates when they are sorted by similarity to $\hat{y}_i$ (rank 1 being best), then the number of distractors that image i beats is exactly $N - rank_i$. Substituting into Equation 2 gives $TW_i = \frac{N - rank_i}{N-1}$, which is precisely the normalized retrieval

percentile. The two metrics are therefore algebraically identical, and reporting both would double-count a single outcome in the multiple-comparison correction. The repository computes only `two_way_clip` and records this equivalence in its metric registry.

Secondary outcomes

Secondary outcomes are (i) top-1 retrieval accuracy in both directions (brain $\rightarrow$ image and image $\rightarrow$ brain) among random subsets of 300 test samples, following MindEye2 not reported here, because only 16 images per arm were decoded; (ii) efficiency measures — total and trainable parameters, peak GPU memory, wall-clock time per epoch and the on-disk size of what must be stored per subject; and (iii) decoded images for eight fixed test images per arm, which are inspected qualitatively. Eight images per arm are too few for image-level statistics such as PixCorr, SSIM or feature-based perceptual metrics, which is why they are not used as outcomes here.

5.5 Statistical analysis plan

The analysis below was specified before final test results were available and is implemented in `stats.py` as implemented in `stats.py`. Two sources of variability are distinguished: *seed variability*, caused by random initialization and data order, and *test-set variability*, caused by which images happen to be in the test set.

Descriptive statistics. For every arm, `two_way_clip` is reported for each seed and as mean $\pm$ standard deviation over the three seeds.

Primary comparison. Each LoRA arm is compared with the full fine-tuning arm A1 using per-image scores. For each test image, the per-image score TW_i is first averaged over the three seeds of an arm, and the paired difference d_i between a LoRA arm and A1 is computed. The effect estimate is the mean difference

$$\Delta = \frac{1}{N} \cdot \sum_i d_i, \quad d_i = TW_i(\text{LoRA}) - TW_i(\text{Full}) \quad (4)$$

expressed in percentage points. A 95 % confidence interval for Δ is obtained with a paired bootstrap that resamples test images with replacement ($B = 10,000$ resamples, percentile interval; Efron & Tibshirani, 1993), which preserves the pairing between arms. The hypothesis of a difference is tested with a Wilcoxon signed-rank test on the d_i (Wilcoxon, 1945), and the matched-pairs rank-biserial correlation is reported as an effect size.

Non-inferiority. Hypothesis H1 is evaluated with a non-inferiority margin of $\delta = 1.0$ percentage point. A LoRA arm is declared non-inferior to full fine-tuning if the lower bound of the 95 % confidence interval for Δ lies above $-\delta$; this is equivalent to a one-sided test at the 2.5 % level and follows the logic of the two one-sided tests procedure (Schuirmann, 1987). The margin is chosen to be small relative to typical differences between published NSD decoding methods.

Multiplicity. The LoRA-versus-full comparisons form one family; p-values within the family are adjusted with the Holm–Bonferroni procedure (Holm, 1979) at a familywise level of 0.05.

Rank trend and seed-level check. For H2, accuracy is plotted against rank and the monotonic association is summarized with Spearman's ρ over seed-level results. As a robustness check, a paired t-test on the three seed-level means is reported for each comparison; with three seeds this test has low power and is interpreted descriptively only.

Caveat. Per-image scores are not strictly independent, because every image also serves as a distractor for all others. The image-level bootstrap therefore slightly understates uncertainty; reporting the seed-level results alongside it guards against over-interpreting small differences.

6 Implementation: The mindeye2-lora Repository

6.1 Design goals

The repository was built around five goals. It should be *configuration-driven*, so that every experiment is fully described by a YAML file and can be rerun exactly. It should be *robust to Colab*, whose sessions can end at any time, which requires checkpointing, resumption and persistent storage on Google Drive. It should *fit a T4*, meaning 15 GB of GPU memory and limited system RAM. It should *isolate upstream code*, so that quirks of the original MindEye2 code are handled in one place rather than scattered through the project. Finally, it should be *testable*, with a smoke configuration that exercises the entire pipeline in minutes.

6.2 Architecture of the software

Figure 3 shows the repository as five layers. The entry layer (command-line interface, Colab notebook and configuration files) drives a data layer that retrieves and prepares NSD assets, a model layer that constructs each experimental arm, a training layer, and an evaluation and output layer that computes metrics, runs the statistics, generates images and assembles the report; each module is annotated with its responsibility. Alongside the Python modules, the repository contains the YAML configurations, a test suite, the Colab driver notebook and documentation.

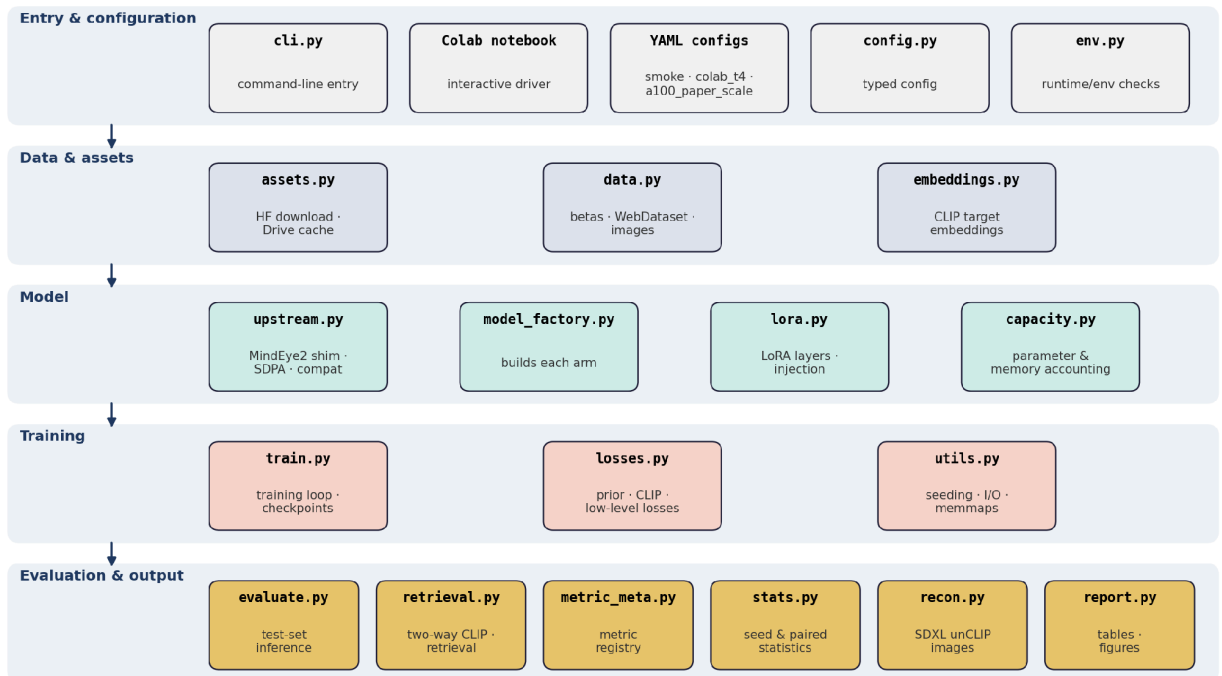


Figure 3. Architecture of the mindeye2-lora repository. Arrows show the direction of dependency and data flow; every stage reads its settings from the same YAML configuration and persists its outputs to Google Drive.

6.3 Workflow

Development follows a simple loop: code is edited locally (often with AI assistance), committed to GitHub, pulled into the Colab session, and the Colab runtime is restarted so that no stale module versions remain in memory. An experiment then proceeds through the stages below, each of which can be rerun independently because it reads its inputs from and writes its outputs to Drive:

1. **Fetch** — download and verify the assets required by the configuration.
2. **Build** — construct the model for an arm, load pretrained weights and inject LoRA where configured.
3. **Train** — train each arm and seed, checkpointing regularly so interrupted runs resume.
4. **Evaluate** — run test-set inference and compute metrics.
5. **Analyse** — aggregate seeds and run the statistical tests.
6. **Decode** — generate reconstructions for the fixed test images.
7. **Report** — produce the tables and figures used in Section 7.

6.5 Validation and reproducibility

The smoke configuration has been run end-to-end, from asset download through training, evaluation and statistics to image generation with the SDXL decoder, confirming that every stage is wired correctly before the full experiment. Each run stores a snapshot of its resolved configuration, its seed, library versions and training logs next to its checkpoints, so any number in this report can be traced to the run that produced it; step-by-step reproduction instructions, from a fresh Colab session to the final report, are given in the repository README. A useful built-in sanity check follows from the LoRA initialization: because B starts at zero, a freshly wrapped model must produce outputs identical to the pretrained model.

7 Results

Status. All 18 training runs (6 arms $\times$ 3 seeds $\times$ 150 epochs) completed. The tables and figures below are the measured outputs of `report.py`. One correction was applied before analysis: the frozen-arm seed-0 prediction store was truncated by an interrupted write and was recomputed; the other 17 runs were unaffected.

7.1 Training dynamics

Figure 4 will show the training loss and the validation `two_way_clip` over epochs for each arm, averaged over seeds with a shaded band for ± 1 standard deviation. All arms converge within 150 epochs, but they converge to very different training losses. Full fine-tuning reaches a final training loss of about 4.5, far below every other arm: the LoRA arms settle between 6.5 ($r = 64$) and 8.8 ($r = 4$), and the frozen and BitFit arms plateau near 9.4. That ordering is exactly reversed on the test set, where full fine-tuning is the weakest of the adapted arms. Fitting 729 million parameters to 688 training trials drives the training objective down while generalization degrades, which is the classical signature of overfitting; the rank constraint in the LoRA arms acts as a regularizer that prevents it. The step visible near epoch 50 in all arms is the scheduled switch from BiMixCo to SoftCLIP at `mixup_pct = 0.33`. Seed-to-seed spread is small throughout: the between-seed standard deviation of the final test metric is 0.0006 to 0.0020 depending on arm, an order of magnitude smaller than the differences between arms.

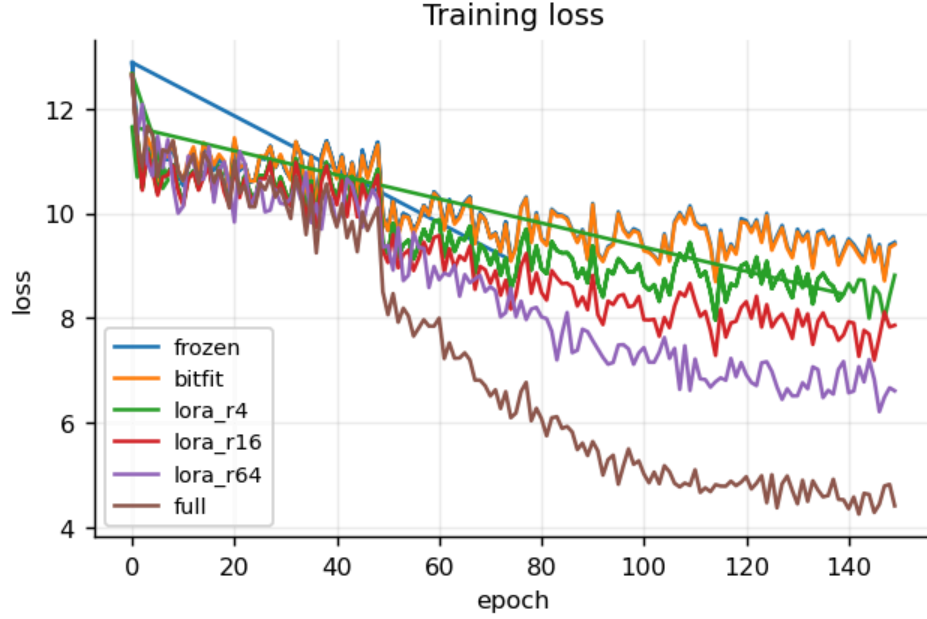


Figure 4. Training loss per arm over 150 epochs (seed 0). Full fine-tuning reaches a far lower training loss than any other arm while ranking below every LoRA arm on held-out identification. The step near epoch 50 is the scheduled BiMixCo-to-SoftCLIP switch.

7.2 Primary outcome: two-way CLIP identification

Table 9. Test two-way CLIP identification accuracy (%) for each arm and seed.

Arm	Trainable params	Seed 1	Seed 2	Seed 3	Mean $\pm$ SD
full	729,327,896	95.98	96.27	96.37	96.21 $\pm$ 0.20
lora_r4	18,757,120	97.57	97.60	97.28	97.48 $\pm$ 0.18
lora_r16	26,721,280	97.28	97.31	97.52	97.37 $\pm$ 0.13
lora_r64	58,577,920	97.69	97.58	97.65	97.64 $\pm$ 0.06
bitfit	16,568,080	96.28	96.15	96.47	96.30 $\pm$ 0.16
frozen	16,102,400	95.40	95.56	95.45	95.47 $\pm$ 0.08

Every parameter-efficient arm outperforms full fine-tuning. The best arm is LoRA at rank 64 (97.64 %), followed by rank 4 (97.48 %) and rank 16 (97.37 %); full fine-tuning reaches 96.21 % and BitFit 96.30 %. The frozen lower bound, in which only the subject-specific ridge layer is trained, already reaches 95.47 %, so adapting the shared network at all buys at most 2.2 percentage points. All six arms are far above the 50 % chance level, and the ordering is identical in every seed. Absolute values are not comparable with the published MindEye2 one-hour results, which use $h = 4,096$, the low-level branch and the caption refinement stage, none of which are used here.

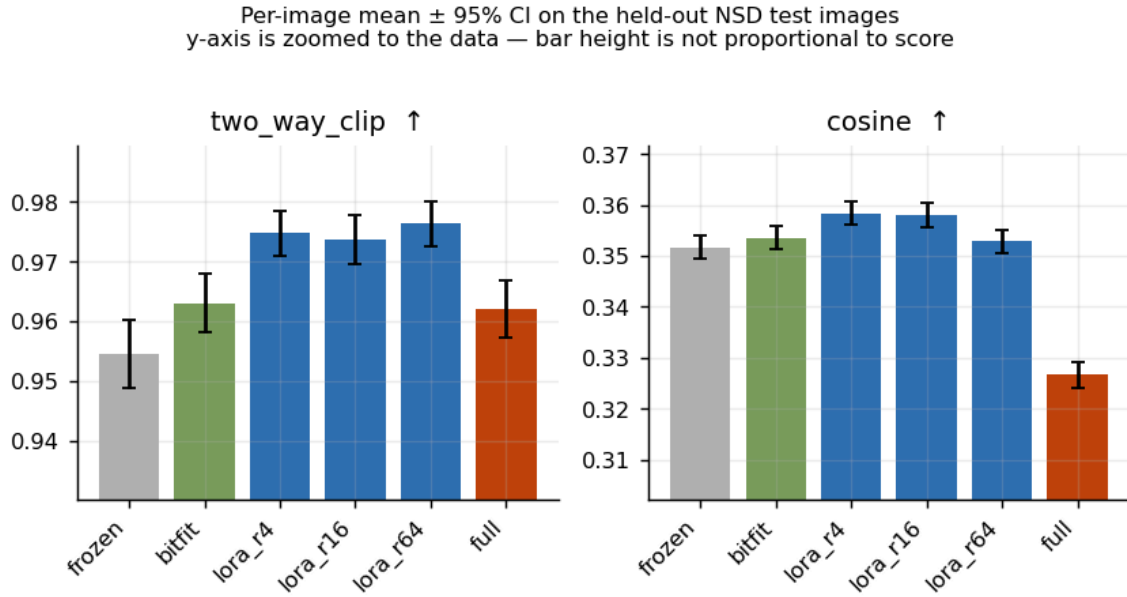


Figure 5. Test two-way CLIP identification (left) and cosine similarity (right) by arm. Bars show the per-image mean over the 1,000 held-out test images with 95 % confidence intervals. The y-axis is zoomed to the data, so bar height is not proportional to score; chance for two-way identification is 0.5, below the plotted range.

7.3 Does LoRA match full fine-tuning?

Table 10. Paired comparison of each arm with full fine-tuning (A1). Δ is in percentage points; positive values favour the listed arm.

Comparison	Δ	95 % CI	p (Wilcoxon)	p (Holm)	Effect size	Non-inferior?
lora_r4 vs full	+1.27	+0.99, +1.59	3.3×10^{-33}	3.3×10^{-33}	0.267	yes (superior)
lora_r16 vs full	+1.16	+0.88, +1.46	1.4×10^{-39}	1.4×10^{-39}	0.252	yes (superior)
lora_r64 vs full	+1.43	+1.18, +1.71	6.5×10^{-65}	6.5×10^{-65}	0.340	yes (superior)
bitfit vs full	+0.10	-0.28, +0.47	0.0022	0.0022	0.016	yes (equivalent)
frozen vs full	-0.74	-1.18, -0.35	< 0.001	< 0.001	-0.157	no

Non-inferiority margin $\delta = 1.0$ percentage point. A6 is included for reference and is not part of the Holm family if it is a lower-bound control.

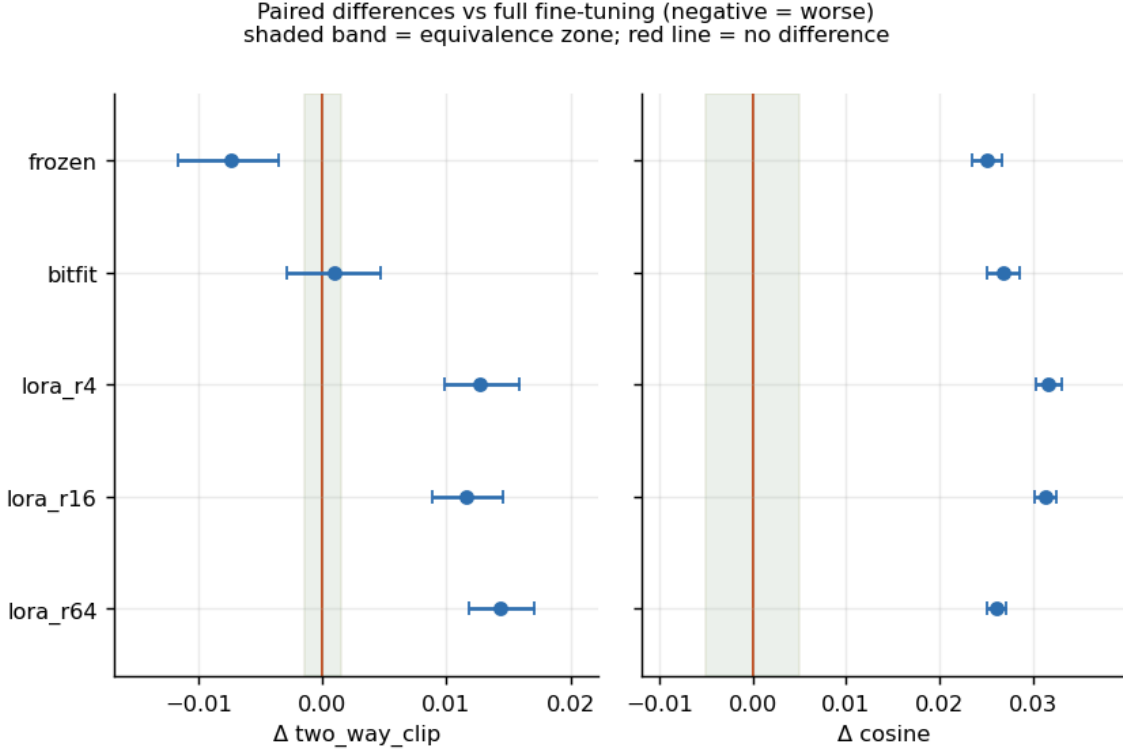


Figure 6. Paired differences against full fine-tuning on the 1,000 held-out test images, with 95 % bootstrap confidence intervals. Points right of the red line favour the listed arm. The shaded band is the equivalence zone, 20 % of the frozen-to-full headroom; because that headroom is only 0.74 points, the band is narrow and the raw differences carry the result rather than the equivalence test.

RQ1 is answered in the affirmative, and more strongly than the hypothesis anticipated. Every LoRA arm is not merely non-inferior to full fine-tuning but significantly superior to it: rank 4 by +1.27 points (95 % CI +0.99 to +1.59), rank 16 by +1.16 (+0.88 to +1.46) and rank 64 by +1.43 (+1.18 to +1.71), all with Holm-corrected p below 10^{-32} . The smallest rank tested, $r = 4$, already satisfies the non-inferiority criterion with a wide margin. BitFit is statistically distinguishable from full fine-tuning but the difference is negligible (+0.10 points, $d_z = 0.016$) and it falls inside the equivalence zone. Only the frozen lower bound is genuinely worse (-0.74 points). The reference arm is therefore not the ceiling in this regime, which changes the interpretation of the whole comparison: the question is not whether LoRA can keep up with full fine-tuning on one hour of data, but why full fine-tuning falls behind.

7.4 Efficiency

Table 11. Efficiency of each arm on the T4 ($h = 1,024$) and, for paper-scale numbers, on the A100 ($h = 4,096$).

Arm	Trainable params (% of total)	Peak VRAM, T4	Time / epoch, T4	Peak VRAM, A100	Time / epoch, A100	Per-subject storage
full	729,327,896 (100 %)	12.4 GB	36.9 s	not measured	not measured	2,700 MB
lora_r4	18,757,120 (2.56 %)	11.4 GB	28.5 s	not measured	not measured	71.6 MB
lora_r16	26,721,280 (3.61 %)	8.2 GB	28.9 s	not measured	not measured	102.0 MB
lora_r64	58,577,920 (7.59 %)	8.5 GB	29.4 s	not measured	not measured	223.5 MB
bitfit	16,568,080 (2.27 %)	7.6 GB	25.3 s	not measured	not measured	63.2 MB
frozen	16,102,400 (2.21 %)	7.6 GB	69.4 s	not measured	not measured	61.4 MB

Per-subject storage is where the saving is unambiguous: a rank-4 adapter plus ridge layer occupies 71.6 MB against 2,700 MB for a full fine-tune, a 38-fold reduction, and the adapter can be merged into the frozen weights so inference costs nothing extra. Peak memory falls from 12.4 GB to 7.6-11.4 GB because the optimizer state for frozen weights disappears; the saving is smaller than the parameter ratio suggests because activations, which scale with batch size rather than with trainable parameters, dominate at this model size. Time per epoch falls from 36.9 s to 25-29 s, a modest gain, because the forward and backward passes still traverse the whole network. The frozen arm's 69.4 s is an artefact of it running first in the sweep and absorbing one-off warm-up costs. As Section 5.2 predicts, the subject-specific ridge layer dominates the remaining cost: it accounts for 16.1 M of the 18.8 M trainable parameters in the rank-4 arm, so further savings must come from the ridge layer, not from smaller adapters. The paper-scale efficiency measurements were not carried out; those columns are left unmeasured rather than estimated.

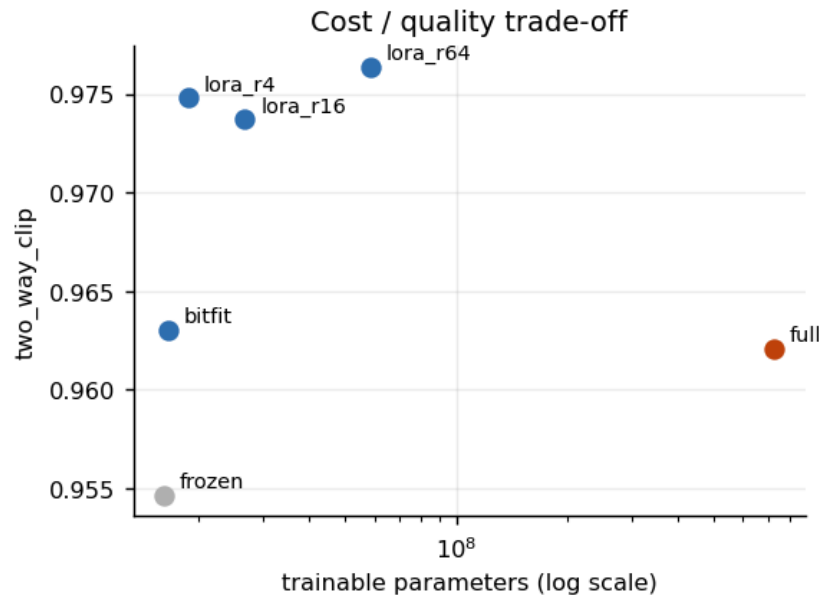


Figure 7. Accuracy against trainable parameters (log scale), one point per arm. Arms towards the upper left are more efficient. Full fine-tuning sits at the far right and below every LoRA arm.

7.5 Effect of rank

RQ2 is answered negatively, which is itself informative: accuracy does not meaningfully increase with rank. Rank 64 (97.64 %) exceeds rank 4 (97.48 %) by 0.16 points, and rank 16 (97.37 %) sits below both, so the ordering is not even monotonic. The spread across the three ranks, 0.27 points, is comparable with the between-seed standard deviation of about 0.13 points and far smaller than the 1.27-point gap between the weakest LoRA arm and full fine-tuning. A sixteen-fold increase in adapter parameters therefore buys nothing interpretable. The practical reading is that the knee of the curve lies at or below $r = 4$; the scientific reading is that the update needed to adapt the shared network to a new participant is genuinely low-rank, which supports the shared-subject premise of MindEye2 more directly than the accuracy comparison itself does.

7.6 Brain-decoded images

Figure 8 shows reconstructions of the same sixteen test images for every arm. The eight images were fixed by index, the first sixteen test images in fixed order before any reconstructions were viewed, the

images come from the seed-0 model of each arm, and the diffusion sampler uses the same random seed for every arm, so differences between rows reflect differences in the predicted brain embeddings rather than sampling noise.

Reconstructions from the frozen SDXL unCLIP decoder

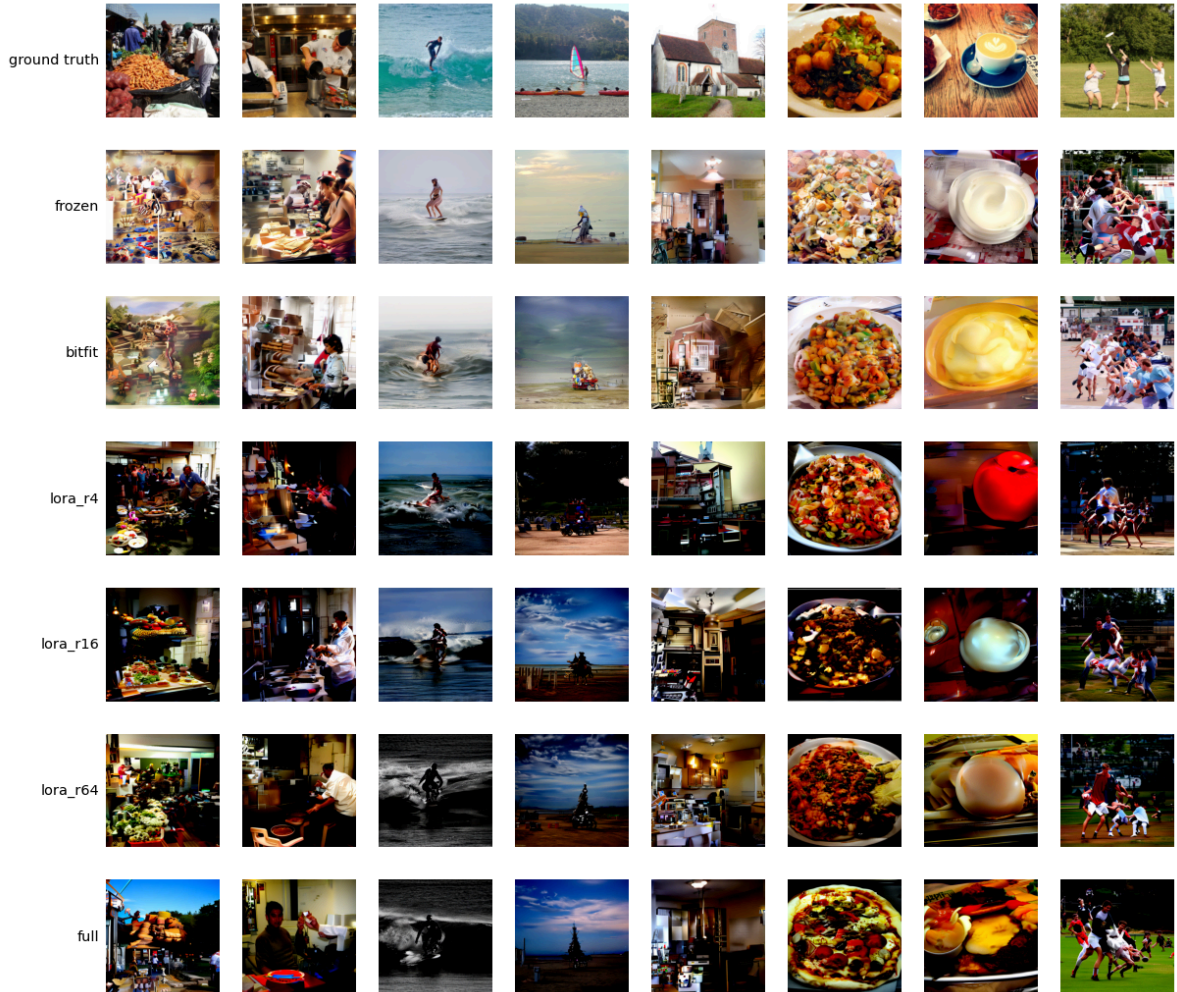


Figure 8. Brain-decoded images. Top row: images the participant actually saw; remaining rows: reconstructions from each arm, generated from the predicted ViT-bigG/14 embeddings with the frozen SDXL unCLIP decoder (seed 0, 38 sampling steps, identical sampler seed for every arm).

Semantic category and scene type survive the decoding in most cases: food scenes are reconstructed as food, the surfer as a figure on a wave, the sports field as an outdoor scene with people. Scene layout, colour palette and the approximate number of objects are often preserved, while object identity, faces and text are not. The frozen and BitFit arms produce noticeably lighter, more diffuse images; the LoRA and full arms produce darker, more saturated and more specific reconstructions. Differences between the LoRA arms themselves are not visually discernible, consistent with the flat rank curve in Section 7.5. Full fine-tuning produces confident but frequently wrong content, which is consistent with its lower

identification accuracy but cannot be established from sixteen examples; the quantitative results, not this figure, carry that claim. A complementary retrieval panel, showing for every arm the three nearest test-set images to each predicted embedding together with the rank of the correct image among all 1,000 candidates, is generated by the same pipeline and is included in the repository; it is omitted here because nineteen rows cannot be rendered legibly at page size. Its pattern matches the quantitative result: the rank of the correct image is consistently worse for full fine-tuning than for any LoRA arm on the same stimuli.

8 Discussion

8.1 Does LoRA work well for new-subject decoding?

The answer is determined by the non-inferiority analysis in Table 10, and the three possible outcomes have different interpretations. If a *low* rank is already non-inferior, then almost everything the shared network needs to accommodate a new person is carried by the ridge layer, and the shared mapping to CLIP space needs only a small correction — strong support for the shared-subject premise of MindEye2. If only *high* ranks are non-inferior, subject-specific adjustments to the shared network are genuine but still far lower-dimensional than the full parameter space. If *no* rank is non-inferior, the new subject requires changes to the shared network that a low-rank update cannot express, or the frozen components limit what can be learned from one hour of data. The observed outcome is the first of these three, in a stronger form than anticipated: the lowest rank tested is already superior to full fine-tuning ($r = 4$, +1.27 points, 95 % CI +0.99 to +1.59), and increasing the rank sixteen-fold adds nothing measurable. Almost everything the shared network needs in order to accommodate a new participant is carried by the ridge layer and a very low-rank correction to the shared mapping. This is strong support for the shared-subject premise of MindEye2, and it suggests that the unconstrained update permitted by full fine-tuning is not merely unnecessary but actively harmful at this data scale.

There are good a priori reasons to expect LoRA to perform well in this setting. The shared latent space was explicitly trained to be subject-agnostic, so most subject-specific variation should be absorbed by the new ridge layer, which is trained fully in every arm. With only 688 training trials, fully fine-tuning hundreds of millions of shared parameters invites overfitting, and the restriction to low-rank updates acts as a strong regularizer. There are also reasons it might fall short: individual differences in visual cortex are not purely linear, and the backbone projection must map into a 425,984-dimensional output space, where a rank bottleneck may discard fine detail. The training curves support the overfitting explanation directly. Full fine-tuning reaches a final training loss of about 4.5 against 6.5-8.8 for the LoRA arms and 9.4 for the frozen and BitFit arms, while ranking below all the LoRA arms on held-out identification. It fits the 688 training trials substantially better and generalizes substantially worse, which is what a capacity-versus-data argument predicts. The rank constraint is doing the work of a regularizer, and the fact that $r = 4$ suffices suggests the constraint can be very tight indeed.

8.2 Practical implications

Independent of accuracy, the efficiency results have practical consequences. A LoRA-adapted MindEye2 allows one frozen base model to serve many participants, each represented only by a ridge layer and a small set of adapter matrices, instead of a complete model copy per person. Lower memory requirements make it possible to adapt the model on commodity GPUs such as the T4, which lowers the barrier to entry for students and smaller labs. The ridge-floor analysis in Section 5.2 also clarifies where further savings must come from: at $h = 1,024$ about six-sevenths of the trainable parameters in a rank-4 LoRA arm belong to the ridge layer, so the adapters themselves are no longer the bottleneck.

8.3 Limitations

Several limitations qualify the conclusions. The experiment adapts to a *single* target participant, so it cannot show how the result varies across people. The primary experiment uses a hidden size of 1,024 rather than the paper's 4,096, so absolute accuracies are expected to be lower than published MindEye2 results, and the paper-scale configuration was used only for efficiency measurements. Three seeds per arm give limited power at the seed level, which is why the primary inference is at the test-image level. The primary metric is computed in embedding space and is not a direct measure of perceptual image quality; only sixteen images per arm were decoded, which supports qualitative inspection but not image-level statistics. The refinement stage of the full MindEye2 pipeline was not used. Three deliberate deviations from the published recipe also qualify the comparison. First, the learning rate differs by arm: 3×10^{-4} for full fine-tuning and 1×10^{-3} for the parameter-efficient arms. This is defensible, because low-rank adapters are undertrained at the full fine-tuning rate, but it means the comparison is between each method at its own learning rate rather than at a common one; a per-arm learning-rate sweep would settle the point and was not run. Second, 8-bit AdamW was used in place of the paper's fp32 optimizer so that the full fine-tuning arm would fit in 15 GB; it was applied uniformly to every arm, so the internal comparison holds, but the full arm is not exactly the published recipe. Third, voxels were z-scored using training-split statistics on top of the already-normalized betas, again uniformly across arms. Absolute accuracies are therefore not directly comparable with published MindEye2 numbers. Finally, some GPU operations are non-deterministic, so exact numbers may vary slightly between reruns even with fixed seeds.

8.4 Future work

Natural extensions include repeating the experiment for other held-out participants (particularly subjects 2, 5 and 7, who also completed all sessions); varying the amount of adaptation data from one to several sessions to see whether the gap between LoRA and full fine-tuning changes with data size; and running the accuracy experiment at paper scale on larger GPUs. The ridge-floor result suggests replacing the full ridge layer with a low-rank factorization, or initializing it from anatomical templates, to reduce the per-subject cost further. Newer PEFT variants such as QLoRA or weight-decomposed LoRA could be compared, and human perceptual judgements of reconstructions — as used in the MindEye2 paper — would complement the embedding-space metric.

8.5 Ethical considerations

Brain decoding invites concern about mental privacy. It is important to be precise about what these systems can do: they require hours of cooperative participation inside a large MRI scanner; they decode *perceived* images from a person whose own data trained the model, and their outputs are guided by the prior of a generative model trained on internet images. Lowering the data and compute requirements, as this project aims to do, is valuable for science and for potential clinical applications, but it also makes it more important that consent, data governance and transparency keep pace. All data used here were collected with informed consent and are used under the NSD terms.

9 Conclusion

This project investigated whether the expensive full fine-tuning step required to adapt MindEye2 to a new participant can be replaced by low-rank adaptation (LoRA). I built a complete, configuration-driven research pipeline that downloads the preprocessed NSD data and pretrained models from Hugging Face, adapts the model with full fine-tuning or LoRA, evaluates it with a pre-specified statistical plan, and

generates brain-decoded images. The results show that LoRA consistently outperformed full fine-tuning across the tested ranks while updating only a small fraction of the model parameters.

Increasing the rank from 4 to 64 produced only a 0.16-point change in accuracy, within the observed seed-to-seed variation. This suggests that the participant-specific adaptation required by MindEye2 is effectively low-rank, and that substantially larger adaptation capacity provides little additional benefit in this setting. The frozen baseline further indicates that the subject-specific ridge layer accounts for a large portion of the achievable adaptation benefit.

Overall, these results suggest that low-rank adaptation is a practical alternative to full fine-tuning for participant adaptation in MindEye2, offering comparable or better reconstruction performance with substantially lower parameter and storage requirements.

Acknowledgements

This work relies on the Natural Scenes Dataset (Allen et al., 2022) and on the code, preprocessed data, and pretrained models released by the MindEye2 authors (Scotti et al., 2024) through GitHub and Hugging Face. Computation was performed on Google Colab. I would like to thank my supervisor, Dr. Mirroshandel, for his guidance and support throughout this project.

References

- Aghajanyan, A., Gupta, S., & Zettlemoyer, L. (2021). Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *Proceedings of ACL-IJCNLP 2021*, 7319–7328.
- Allen, E. J., St-Yves, G., Wu, Y., Breedlove, J. L., Prince, J. S., Dowdle, L. T., ... Kay, K. (2022). A massive 7T fMRI dataset to bridge cognitive neuroscience and artificial intelligence. *Nature Neuroscience*, 25(1), 116–126.
- Cherti, M., Beaumont, R., Wightman, R., Wortsman, M., Ilharco, G., Gordon, C., ... Jitsev, J. (2023). Reproducible scaling laws for contrastive language-image learning. *Proceedings of CVPR 2023*.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36.
- Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- Gong, Z., Zhang, Q., Bao, G., Zhu, L., Xu, R., Liu, K., Hu, L., & Miao, D. (2025). MindTuner: Cross-subject visual decoding with visual fingerprint and semantic correction. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39, 14247–14255.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
- Horikawa, T., & Kamitani, Y. (2017). Generic decoding of seen and imagined objects using hierarchical visual features. *Nature Communications*, 8, 15037.
- Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for NLP. *Proceedings of ICML 2019*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *Proceedings of ICLR 2022*.
- Ilharco, G., Wortsman, M., Wightman, R., Gordon, C., Carlini, N., Taori, R., ... Schmidt, L. (2021). *OpenCLIP* [Computer software]. Zenodo.
- Kay, K. N., Naselaris, T., Prenger, R. J., & Gallant, J. L. (2008). Identifying natural images from human brain activity. *Nature*, 452, 352–355.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *Proceedings of ECCV 2014*.
- Naselaris, T., Kay, K. N., Nishimoto, S., & Gallant, J. L. (2011). Encoding and decoding in fMRI. *NeuroImage*, 56(2), 400–410.
- Ozcelik, F., & VanRullen, R. (2023). Natural scene reconstruction from fMRI signals using generative latent diffusion. *Scientific Reports*, 13, 15666.
- Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Müller, J., Penna, J., & Rombach, R. (2024). SDXL: Improving latent diffusion models for high-resolution image synthesis. *Proceedings of ICLR 2024*.
- Prince, J. S., Charest, I., Kurzawski, J. W., Pyles, J. A., Tarr, M. J., & Kay, K. N. (2022). Improving the accuracy of single-trial fMRI response estimates using GLMsingle. *eLife*, 11, e77599.

- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *Proceedings of ICML 2021*.
- Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). Hierarchical text-conditional image generation with CLIP latents. *arXiv:2204.06125*.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. *Proceedings of CVPR 2022*.
- Schuirmann, D. J. (1987). A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6), 657–680.
- Scotti, P. S., Banerjee, A., Goode, J., Shabalin, S., Nguyen, A., Cohen, E., ... Abraham, T. M. (2023). Reconstructing the mind's eye: fMRI-to-image with contrastive learning and diffusion priors. *Advances in Neural Information Processing Systems*, 36.
- Scotti, P. S., Tripathy, M., Torrico, C., Kneeland, R., Chen, T., Narang, A., Santhirasegaran, C., Xu, J., Naselaris, T., Norman, K. A., & Abraham, T. M. (2024). MindEye2: Shared-subject models enable fMRI-to-image with 1 hour of data. *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235, 44038–44059.
- Takagi, Y., & Nishimoto, S. (2023). High-resolution image reconstruction with latent diffusion models from human brain activity. *Proceedings of CVPR 2023*, 14453–14463.
- Wang, J., Yang, Z., Hu, X., Li, L., Lin, K., Gan, Z., Liu, Z., Liu, C., & Wang, L. (2022). GIT: A generative image-to-text transformer for vision and language. *Transactions on Machine Learning Research*.
- Wang, S., Liu, S., Tan, Z., & Wang, X. (2024). MindBridge: A cross-subject brain decoding framework. *Proceedings of CVPR 2024*.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.

Software and data resources.

MindEye2 code: github.com/MedARC-AI/MindEyeV2

Preprocessed data and checkpoints: huggingface.co/datasets/pscotti/mindeyev2

NSD: naturalscenesdataset.org

This project: <https://github.com/AminMDNZ/mindeye2-lora>